Machine Learning in a Nutshell

A Physicist's Guide to the Practical Man

Caio Laganá Fernandes

August 5, 2026

Contents

Preface

So, you're a physicist looking to dive into the world of machine learning. You have a deep understanding of calculus, are familiar with experimental data-fitting methods, and have mastered concepts like entropy and non-linearity—yet you have only a vague idea of how a machine learning algorithm is actually written and how it works? Then this book is for you. And the good news is: your learning curve will grow fast. I've been in your shoes. I used to view machine learning as a distant horizon, but I soon realized that the knowledge I'd gained in physics made it incredibly easy to quickly master the concepts and engineering behind the field. So, come along and be amazed.

Chapter 1

Few-neuron networks

Toy models are useless for real-world applications, but are wonderful for didactic purposes. In this chapter, I use few-neuron models to present *the* fundamental concepts for any advanced topic in ML theory.

1.1 Single-neuron ML

You have been using a single-neuron network since the first time you fitted experimental data, but perhaps no one ever told you that. Let's recall the basics. You have a set of N experimental data points

$$(x_i, y_i) \tag{1.1}$$

through which to fit a linear function

$$f(x) = ax + b. \tag{1.2}$$

The Mean Squared Error Loss, defined as

$$\begin{aligned} L_{\text{MSE}}(a,b) &= \frac{1}{N}\sum_{i=1}^{N}[y_i - f(x_i)]^2 \\ &= \frac{1}{N}\sum_{i=1}^{N}[y_i - (ax_i + b)]^2 \end{aligned} \tag{1.3}$$

tells us how distant the curve $f(x)$ is from the datapoints. Fitting $f(x)$ to the datapoints (x_i, y_i) means minimizing $L_{\text{MSE}}(a,b)$ with respect to a and b, that is,

$$\frac{\partial}{\partial(a,b)} L_{\text{MSE}}(a,b) = 0. \tag{1.4}$$

No surprises so far, right? Well, what if I tell you the above is the simplest possible neural network? Indeed, it is a single-neuron network, where (1.2) is (almost!) the activation function, (1.3) is the loss function and (1.4) is the embryo to gradient descendent method.

Solving (1.4) for a and b, which, in this simple case is possible analytically, fixes the parameters to a_0 and b_0

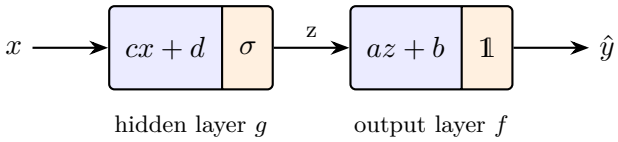
$$\begin{aligned} a_0 &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \\ b_0 &= \bar{y} - a_0 \bar{x} \end{aligned} \tag{1.5}$$

and we end up with a model $f_0(x) = a_0 x + b_0$ (in (1.5), $\bar{x}$ and $\bar{y}$ are the mean values of the x_i and y_i datapoints). This model is a single neuron, no hidden layers: it receives one input parameter and outputs a number. f_0 has learned from data (1.1) through (1.4) and can now predict on any given x. Pretty cool, huh? The concepts were sitting there all the time. Take a moment to re-read the definitions and let's jump to the second simplest model: a two-neuron network.

1.2 Two-neuron ML

Our neuron from Sec. 1.1 was of the form $f(x) = ax + b$ (always think of it as receiving one value x and, upon training for a, b parameters, outputting another value $a_0x + b_0$). Let us now place a second neuron, g, before it, so that g receives the input and f produces the final output. This new neuron will be of the form $g(x) = \sigma(cx + d)$, where $\sigma(x) = 1/(1 + e^{-x})$. The need for σ will be elucidated later; for now, just accept it as a magical ingredient. In fact, it is the neuron's so-called activation function!

Placing one neuron before another means composing the two functions, that is, given input x, the network outputs $f(g(x))$. In other words, input x feeds a (hidden) layer g, which applies its weight c and bias d, activates through σ and feeds output layer f, which in turn applies weight a and bias b and outputs $\hat{y}$. Layer f has identity operator $\mathbb{1}$ as its activation function. A visual diagram may help.



Our task now is to optimize the network for parameters a, b, c, d over training data. The loss function is similar to (1.3), just generalized for the two-neuron case,

$$L_{\text{MSE}}(a, b, c, d) = \frac{1}{N}\sum_{i=1}^{N}[y_i - f(g(x_i))]^2. \tag{1.6}$$

This time, however, MSE minimization, stated as

$$\frac{\partial}{\partial(a, b, c, d)} L_{\text{MSE}}(a, b, c, d) = 0 \tag{1.7}$$

cannot be solved analytically as in (1.5), and numerical methods must be used. The recipe is as follows: start at a certain initial position (a, b, c, d) in the four-dimensional parameter space, then slightly change each coordinate by an amount according to[1]:

$$\begin{aligned} a &\to a - \alpha\frac{\partial L_{\text{MSE}}}{\partial a} \\ b &\to b - \alpha\frac{\partial L_{\text{MSE}}}{\partial b} \\ c &\to c - \alpha\frac{\partial L_{\text{MSE}}}{\partial c} \\ d &\to d - \alpha\frac{\partial L_{\text{MSE}}}{\partial d} \end{aligned} \tag{1.8}$$

The term α is called the learning rate (hallelujah, a name you've heard before, now in a clear context!). The derivatives of L_{MSE} in (1.8) point to the max uphill on the hypersurface, so the minus sign will move the initial coordinate to a new position where L_{MSE} is lower. After repeating this procedure a couple of times, (1.7) will be solved to a certain desired precision (yes, I am oversimplifying, this step is a billion-dollar industry).

The recipe just described for adjusting (a, b, c, d) parameters based on the partial derivatives (1.8) is called *gradient descent*. The computations of the derivatives themselves will lead to the

[1] Picture this process as finding a minimum on a graph. For reference, in Sec. 1.1 the graph of the loss function, $(a, b, L_{\text{MSE}}(a, b))$, was a two-dimensional paraboloid embedded in $\mathbb{R}^3$; in this two-neuron scenario, the graph is a four-dimensional hypersurface embedded in $\mathbb{R}^5$. The paraboloid has a clear, well-defined minimum; this time, there may be several local minima plus saddle points and plateaus, and which one you land in depends on the starting point.

concept of *backpropagation*. Stated like this, with bare partial derivatives and almost no intuition of what is going on behind the scenes, the recipe sounds a bit barren. Let me improve it.

The central idea of gradient descendet and backpropagation is to tell (or, more precisely, to *signal*) the neurons in which direction they must adjust their parameters, and by how much, in order to minimize the final loss. Think about it: how can a hidden neuron (in our two-neuron model, g) possibly gather information about the loss at the very output? This is the central problem addressed. Let's dive a little into the algebra.

The partial derivatives present in (1.8), obtained through chain-rule derivatives on (1.6), can, in this case, be taken explicitly:

$$\begin{aligned}
\frac{\partial L}{\partial a} &= \frac{\partial L}{\partial f}\cdot\frac{\partial f}{\partial a} = -\frac{2}{N}\sum_i \big[y_i - f(g(x_i))\big]g(x_i)\\
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial f}\cdot\frac{\partial f}{\partial b} = -\frac{2}{N}\sum_i \big[y_i - f(g(x_i))\big]\\
\frac{\partial L}{\partial c} &= \frac{\partial L}{\partial f}\cdot\frac{\partial f}{\partial g}\cdot\frac{\partial g}{\partial c} = -\frac{2a}{N}\sum_i \big[y_i - f(g(x_i))\big]\sigma'(cx_i+d)x_i\\
\frac{\partial L}{\partial d} &= \frac{\partial L}{\partial f}\cdot\frac{\partial f}{\partial g}\cdot\frac{\partial g}{\partial d} = -\frac{2a}{N}\sum_i \big[y_i - f(g(x_i))\big]\sigma'(cx_i+d)
\end{aligned} \tag{1.9}$$

A suitable reparametrization of variables will be of great algebraic assistance. (We know from other branches of physics—electromagnetism, condensed matter, particle physics—that a "suitable change of variables" often carries deep phenomenological significance. So, hold on!)

$$\begin{aligned}
u_i &= cx_i + d && \text{hidden pre-activation}\\
z_i &= \sigma(u_i) && \text{hidden activation}\\
\hat{y}_i &= f(g(x_i)) && \text{network output}
\end{aligned} \tag{1.10}$$

In terms of (1.10), define the *error signal* as

$$\delta_i = \frac{\partial L}{\partial \hat{y}_i} = -\frac{2}{N}(y_i - \hat{y}_i), \tag{1.11}$$

and the *hidden error signal* as

$$\delta_i^h = \delta_i a\sigma'(u_i). \tag{1.12}$$

As the name suggests, the error signals will *signal* the neurons in which direction to change their parameters, and by how much. With all these new variables, the partials (1.9) read

$$\begin{aligned}
\frac{\partial L}{\partial a} &= \sum_i \delta_i z_i\\
\frac{\partial L}{\partial b} &= \sum_i \delta_i\\
\frac{\partial L}{\partial c} &= \sum_i \delta_i^h x_i\\
\frac{\partial L}{\partial d} &= \sum_i \delta_i^h
\end{aligned} \tag{1.13}$$

Phew! Much cleaner! And in return we've gained an astonishing behaviour: instead of computing all the messy partial derivatives in (1.9), all we need to do in order to calibrate each neuron's parameters is to compute its respective error signal and apply elementary algebra. This shortcut is called *backpropagation*. The name stems from the fact that, starting at the output and going backwards, each crossed layer will render the neuron one error signal. This behaviour showed up explicitly in this two-neuron toy model, but in fact it generalizes to networks of any size. This prepares our terrain for the next chapter.